"Out of many, one"

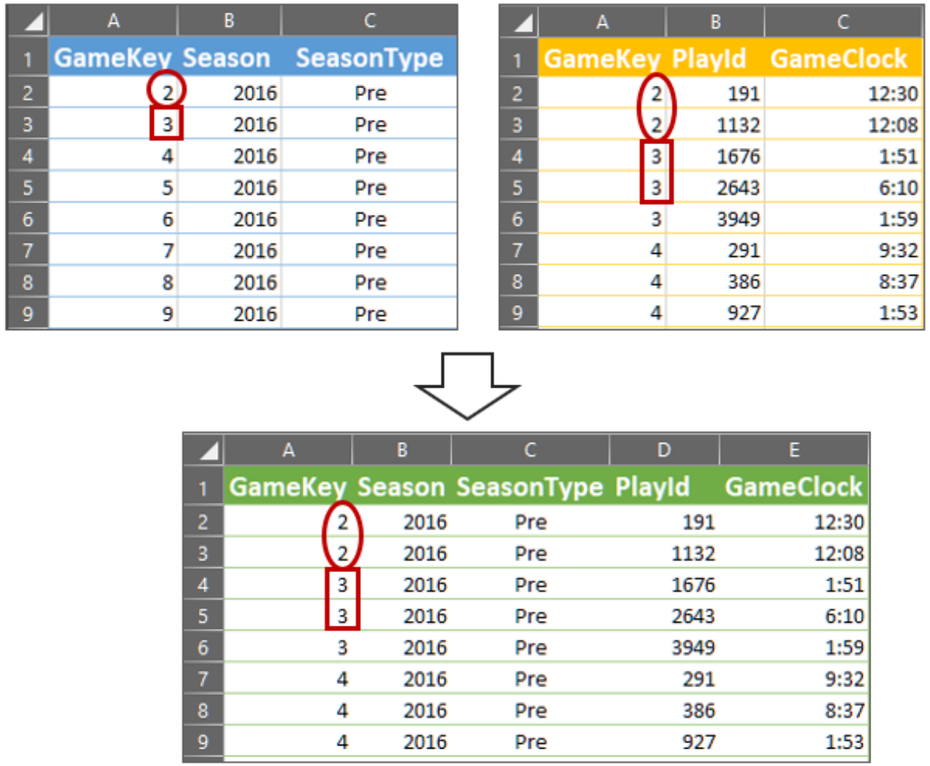
PANDAS JOINS FOR SPREADSHEET USERS

John Miller
Principal Data Scientist

datacamp

One-to-many joins

- One row matches to many rows
- Resulting structure mirrors the lower level



	A	B	C
1	GameKey	Season	SeasonType
2	2	2016	Pre
3	3	2016	Pre
4	4	2016	Pre
5	5	2016	Pre
6	6	2016	Pre
7	7	2016	Pre
8	8	2016	Pre
9	9	2016	Pre

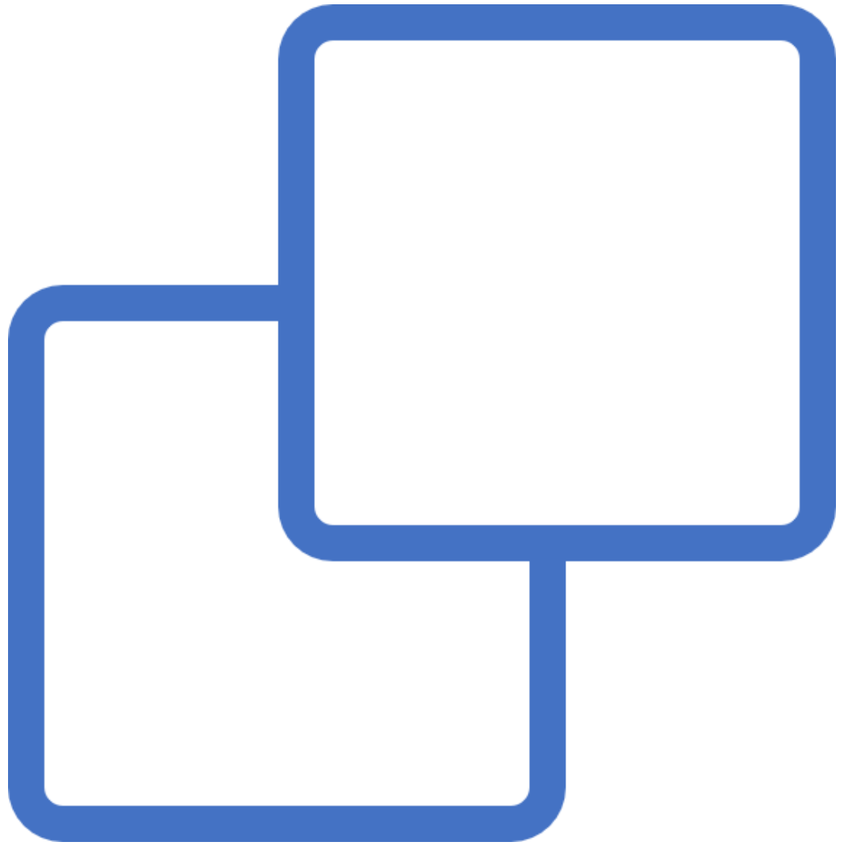
	A	B	C
1	GameKey	PlayId	GameClock
2	2	191	12:30
3	2	1132	12:08
4	3	1676	1:51
5	3	2643	6:10
6	3	3949	1:59
7	4	291	9:32
8	4	386	8:37
9	4	927	1:53

	A	B	C	D	E
1	GameKey	Season	SeasonType	PlayId	GameClock
2	2	2016	Pre	191	12:30
3	2	2016	Pre	1132	12:08
4	3	2016	Pre	1676	1:51
5	3	2016	Pre	2643	6:10
6	3	2016	Pre	3949	1:59
7	4	2016	Pre	291	9:32
8	4	2016	Pre	386	8:37
9	4	2016	Pre	927	1:53

Revisiting the framework

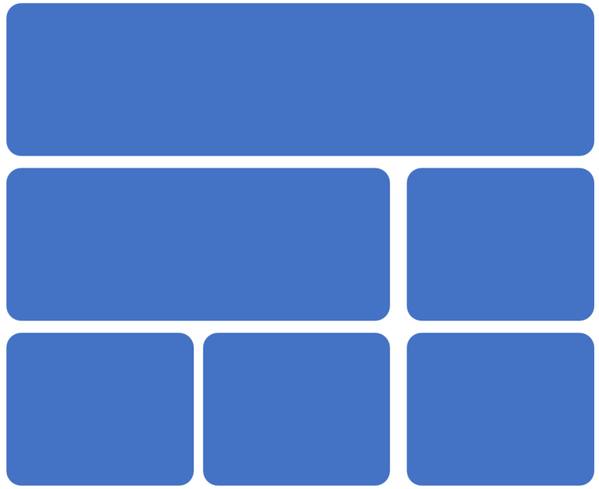
After viewing and understanding the data:

- **Determine the relationship**



Relationship examples

- Time relationships
  - Month and days
  - Hour and minutes
- Hierarchy relationships
  - Company and departments
  - Nations and cities
- Different entities
  - Customer and purchase orders
  - Department descriptions and expense breakdowns



Let's practice!

PANDAS JOINS FOR SPREADSHEET USERS

Joining on key columns

PANDAS JOINS FOR SPREADSHEET USERS

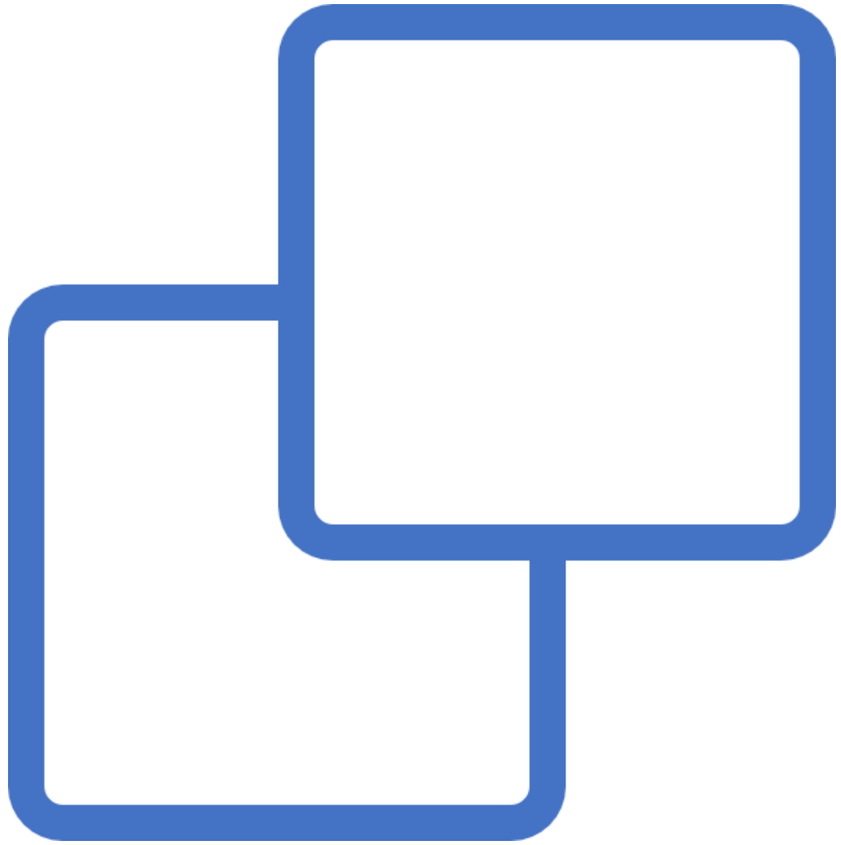
John Miller
Principal Data Scientist

datacamp

Framework (continued)

After viewing and understanding the data:

- Determine the relationship
- **Check for unique values in key column**



Unique key columns

Unique values for single column key

```
df.duplicated('GameKey').sum()
```

-- -- -- A value of 0 means no duplicates -- -- --

```
df.duplicated(['GameKey', 'PlayId').sum()
```

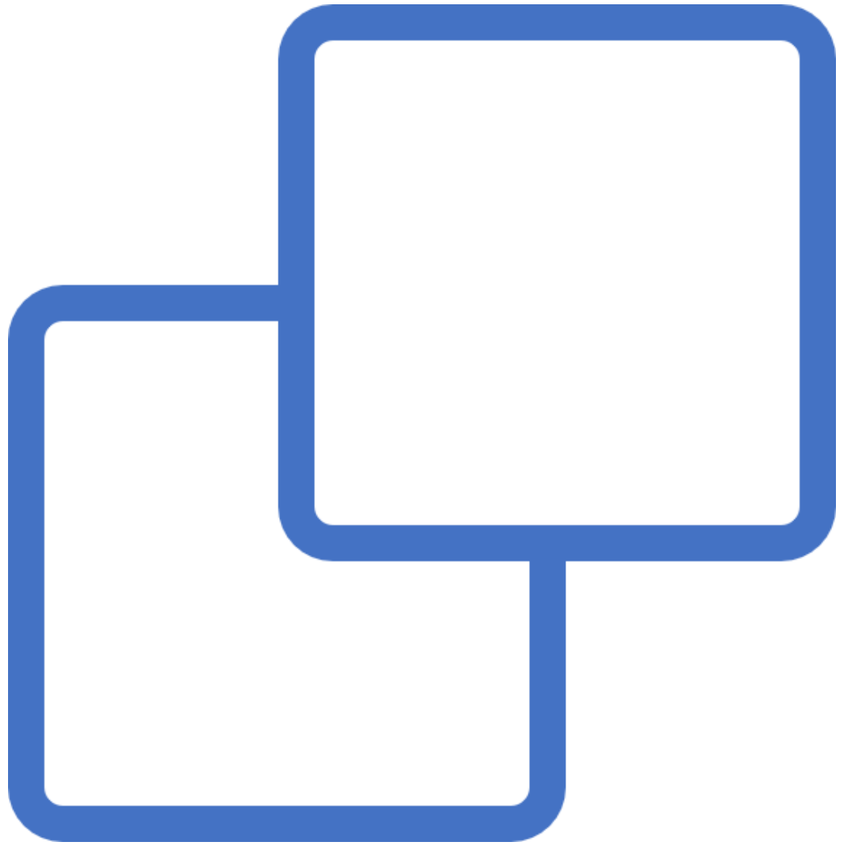
Game_Id	Game_Date	Game_Day	Start_Time
1	8/7/2016	Sunday	20:00
2	8/13/2016	Saturday	17:00
3	8/11/2016	Thursday	19:30
4	8/12/2016	Friday	19:00
5	8/11/2016	Thursday	19:00
6	8/12/2016	Friday	19:00
7	8/14/2016	Sunday	16:00
8	8/13/2016	Saturday	19:00
9	8/11/2016	Thursday	19:30
10	8/12/2016	Friday	19:00

			X	Y
GameId	PlayId	DisplayName		
20170907	118	Eric Berry	73.91	34.84
		Daniel Sorensen	56.63	26.90
		Chris Hogan	76.47	36.91
	139	Justin Houston	66.02	31.50
		Chris Jones	66.80	27.55

Framework (continued)

After viewing and understanding the data:

- Determine the relationship
- Check for unique values in key column
- **Write merge statement and execute**



Executing the merge

The statement is the same!

```
df1.merge(df2, how='inner', on='')
```

- Pay attention to parameters

Full syntax:

```
DataFrame.merge(right, how='inner', on=None, left_on=None, right_on=None,
left_index=False, right_index=False, sort=False, suffixes=('_x', '_y'), copy=True,
indicator=False, validate=None)
```

Validating merges

```
DataFrame.merge(right, how='inner', on=None, left_on=None, right_on=None,
left_index=False, right_index=False, sort=False, suffixes=('_x', '_y'), copy=True,
indicator=False, validate=None)
```

Values for `validate` :

- “one_to_one” or “1:1”
- “one_to_many” or “1:m”
- “many_to_one” or “m:1”
- “many_to_many” or “m:m” (does nothing)

Let's practice!

PANDAS JOINS FOR SPREADSHEET USERS

Index-based joins

PANDAS JOINS FOR SPREADSHEET USERS

John Miller
Principal Data Scientist

datacamp

Pandas indexing

- Generic numbered indexing

generic	GameId	PlayId	DisplayName	X	Y
0	20170907	118	Eric Berry	73.91	34.84
1	20170907	118	Daniel Sorensen	56.63	26.90
2	20170907	118	Chris Hogan	76.47	36.91
3	20170907	139	Justin Houston	66.02	31.50
4	20170907	139	Chris Jones	66.80	27.55

- Tailored multi-level indexing

GameId	PlayId	DisplayName	X	Y
20170907	118	Eric Berry	73.91	34.84
		Daniel Sorensen	56.63	26.90
		Chris Hogan	76.47	36.91
	139	Justin Houston	66.02	31.50
		Chris Jones	66.80	27.55

Joining on index

```
DataFrame.join(other, how='')
```

- Similar to merge on columns
- Joins on index by default
- Can join multiple data frames

```
df.join([df2, df3], how='')
```

Let's practice!

PANDAS JOINS FOR SPREADSHEET USERS